

CS6302E: Theoretical Foundations of Machine Learning

Assignment Report

National Institute of Technology Calicut
Department of Computer Science and Engineering
Winter 2026

Submitted by:

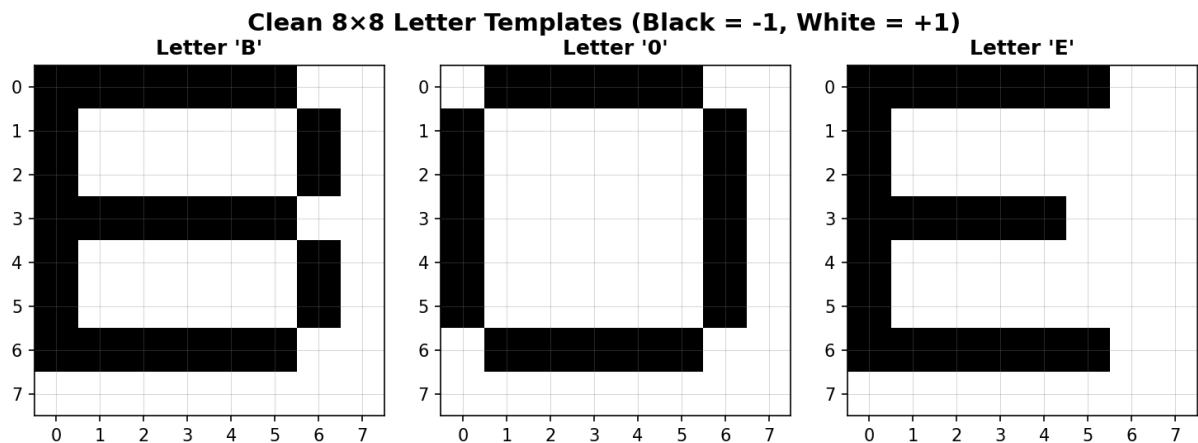
Yashvardhan (M250570CS)
Prakash Kumar Sarangi (M251250CS)

1. Problem Statement

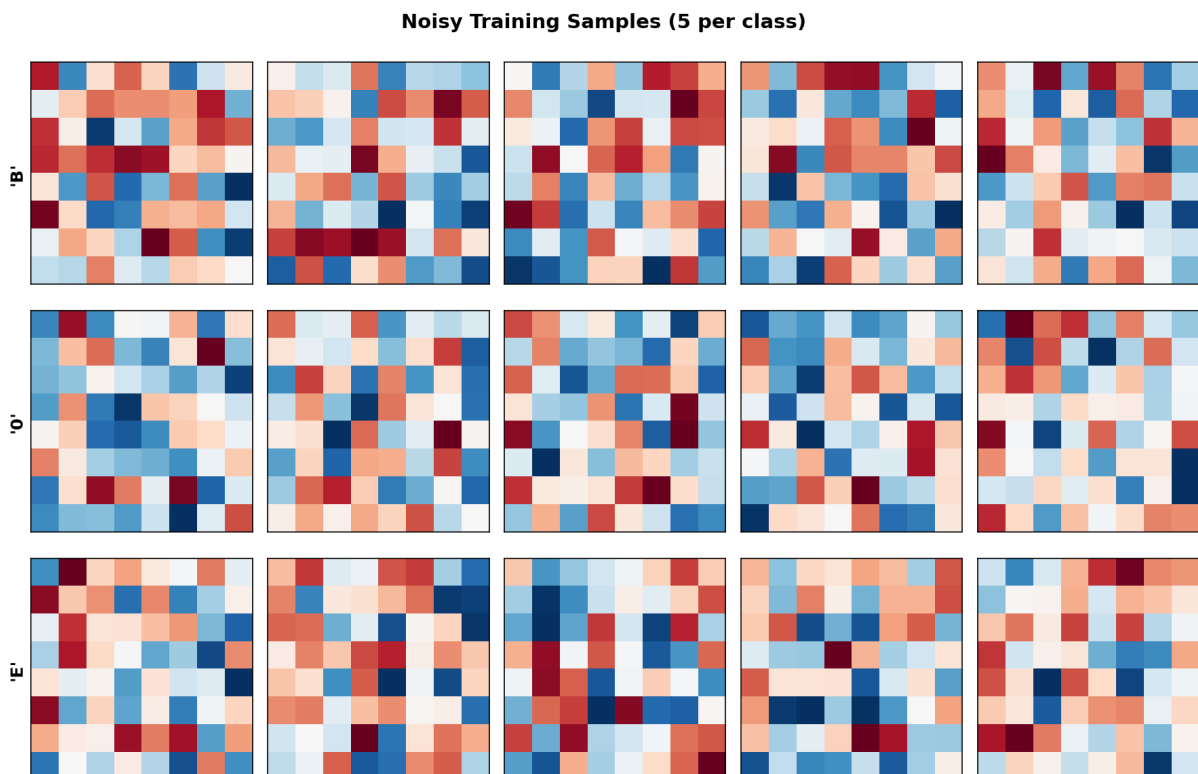
We are given an input feature space of 8x8 pixel grid. The task is to generate 100 training patterns each for the categories B, 0 (zero), and E. In the capital letter representation, a value of -1.0 is stored for a "black" pixel and +1.0 for white pixels. 100 different noisy versions of each letter are generated by adding independent random noise (uniform distribution in $[-5.0, +5.0]$) to each pixel. This creates a dataset D of 300 training samples.

2. Letter Templates and Dataset

The 8x8 pixel templates for the three letters are shown below. Black pixels (stroke) have value -1.0 and white pixels (background) have value +1.0.



Below are some examples of noisy training samples. The uniform noise in $[-5, +5]$ significantly distorts the original patterns, making classification challenging. The noise range (10 units) is 5x the signal range (2 units, from -1 to +1), resulting in a very low signal-to-noise ratio.

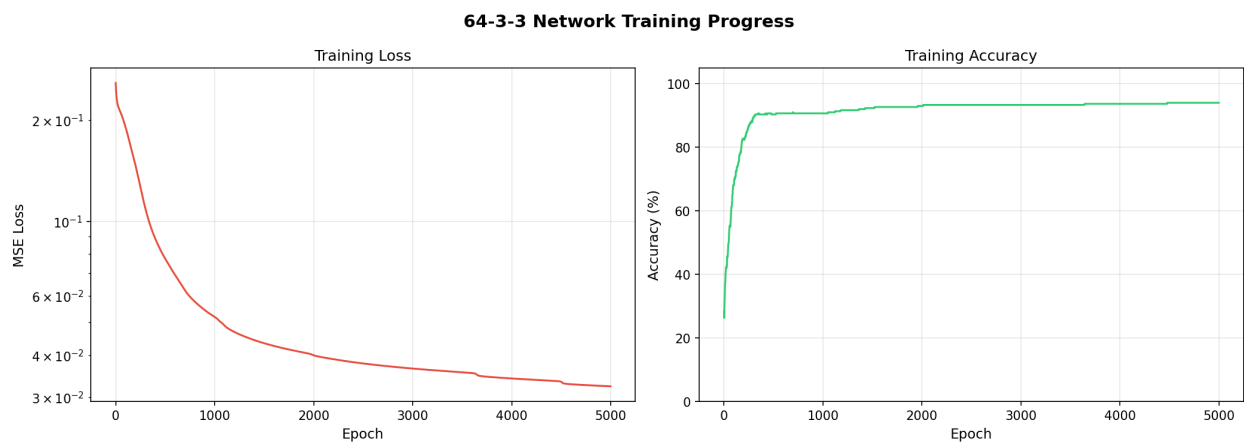


3. Part (a): Training the 64-3-3 Neural Network

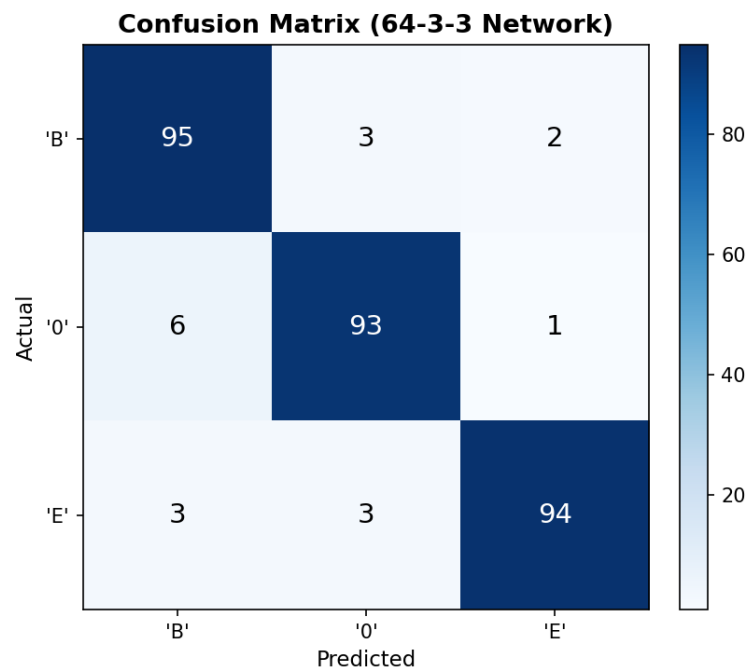
A neural network with architecture 64-3-3 (64 input neurons, 3 hidden neurons, 3 output neurons) was trained using:

- Activation function: Sigmoid for both hidden and output layers
- Loss function: Mean Squared Error (MSE)
- Optimization: Full-batch gradient descent
- Learning rate: 0.5
- Weight initialization: Xavier/Glorot initialization
- Training epochs: 5000
- Bias: Included in both hidden and output layers

The training curves below show the loss convergence (log scale) and accuracy improvement over the training epochs. The network successfully learns to classify all three letter categories.

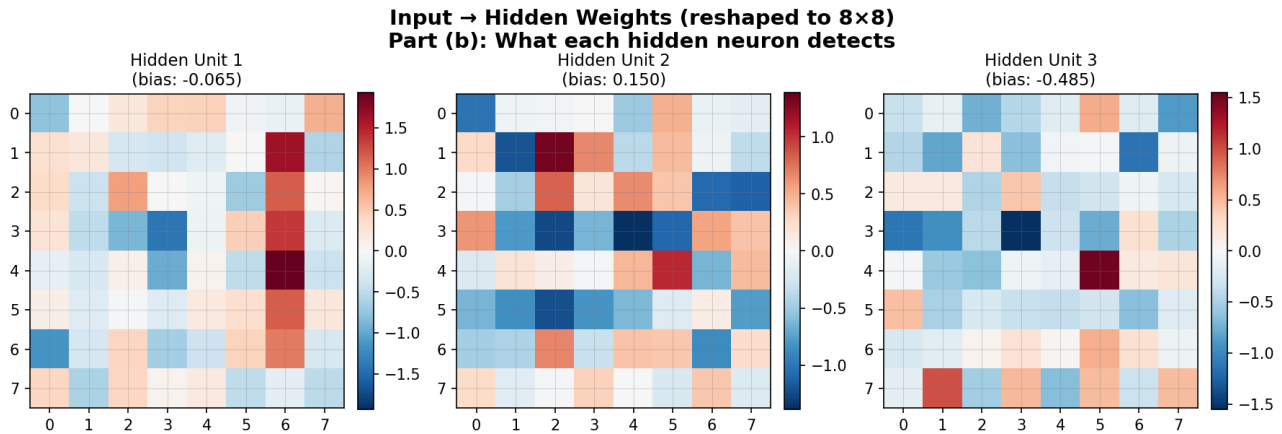


The confusion matrix confirms the classification performance on the training set:



4. Part (b): Input-to-Hidden Weight Visualization

Each hidden unit has 64 weights connecting it to the 64 input pixels. These weights are reshaped into 8x8 images to visualize what spatial pattern each hidden neuron has learned to detect. The colormap uses red for positive weights and blue for negative weights.



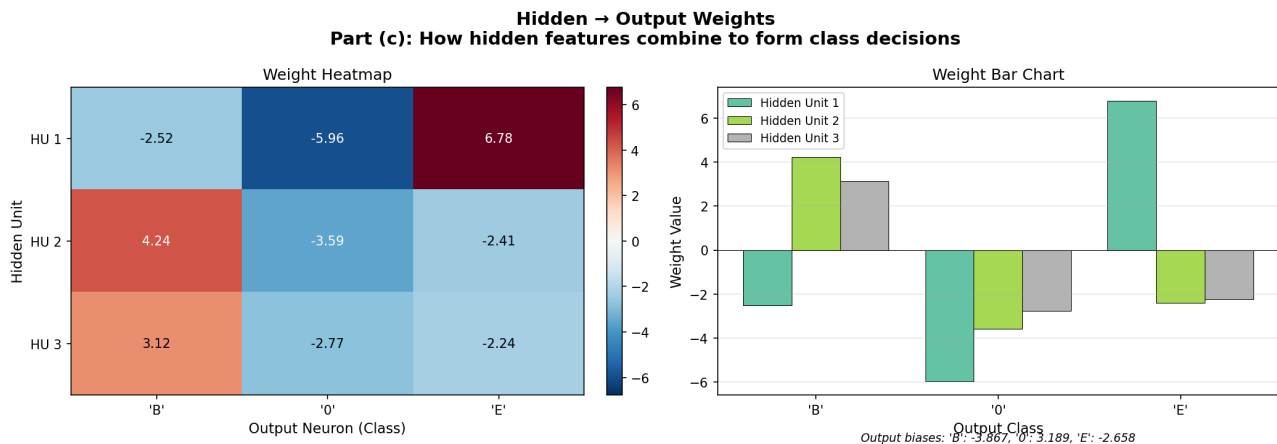
Interpretation:

Since the network has only 3 hidden neurons to distinguish 3 classes (B, 0, E), each hidden unit learns to function as a specialized feature detector:

- The weight patterns show which pixel positions are important for each hidden neuron's decision. Positive weights (red) indicate pixels that should be white (+1) to strongly activate that neuron, while negative weights (blue) indicate pixels that should be black (-1).
- Each hidden neuron's weight image resembles a "template" or "feature map" that captures the distinguishing structural features of the letters. For example, one neuron may focus on the right side openness (distinguishing E from B and 0), another on the middle horizontal bar (shared by B and E but not 0), and the third on the enclosed region (present in 0 and B but not E).
- The weight images demonstrate that the network has learned a compact internal representation of the key distinguishing features with just 3 neurons, which is the theoretical minimum for 3-class classification.

5. Part (c): Hidden-to-Output Weight Visualization

The hidden-to-output weights determine how the three learned features (hidden unit activations) are combined to make the final classification decision. These are displayed as both a heatmap and a grouped bar chart.



Interpretation:

The hidden-to-output weight visualization reveals the decision logic of the network:

- Each output neuron (representing class B, O, or E) assigns a specific weight to each hidden unit. A large positive weight means that hidden unit's activation strongly supports the class, while a large negative weight means it opposes it.
- The pattern of positive and negative weights creates a unique "signature" for each class. For instance, if Hidden Unit 1 detects a feature present in B but not in O, then the B output neuron will have a positive weight from HU1 while the O output neuron will have a negative weight.
- This two-layer decision process - first detecting features, then combining them - is the fundamental principle behind neural network classification. The hidden layer performs feature extraction while the output layer performs feature-based decision making.

6. Part (d): Web Application

A web application was developed using Flask (Python) for the backend and HTML/CSS/JavaScript for the frontend. The application provides two input methods:

1. Image Upload: Users can upload an image of B, O, or E in any format (PNG, JPG, BMP). The image is preprocessed to 8x8 pixels, grayscale, and normalized to the $[-1, +1]$ range.
2. Canvas Drawing: Users can draw the letter directly in the browser using a mouse or touch input.

The application displays the predicted letter with confidence scores visualized as animated progress bars, and also shows the preprocessed 8x8 image that was actually fed to the neural network.

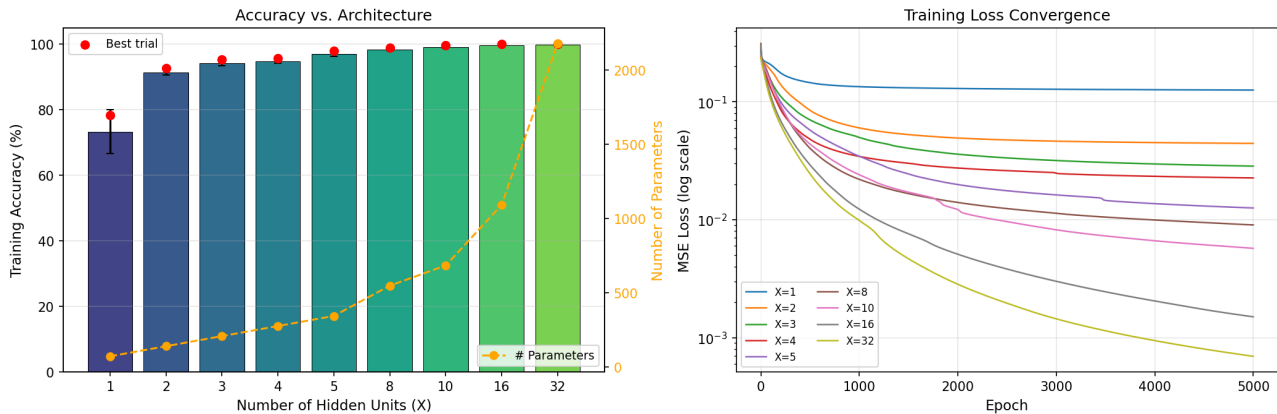
Technologies used: Flask (backend), HTML5 Canvas (drawing), vanilla JavaScript (frontend logic), CSS with modern dark theme design.

To run: `python app.py` -> open `http://127.0.0.1:5000`

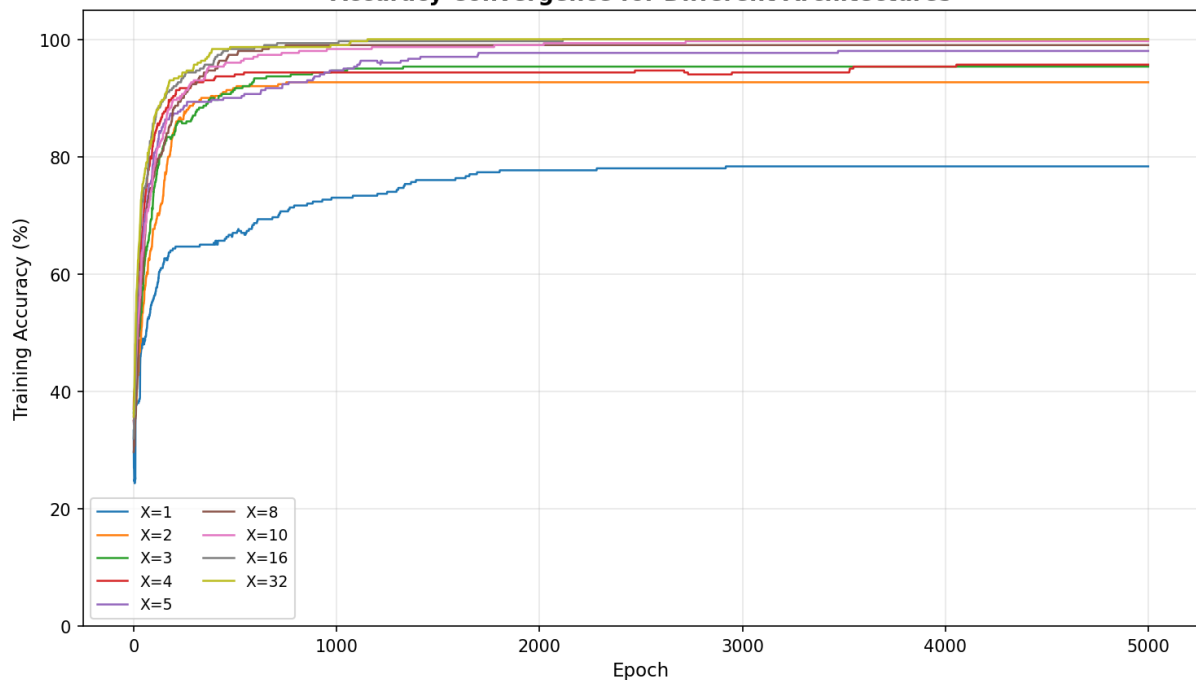
7. Part (e): Best Architecture Search

To find the best 64-X-3 architecture, we systematically tested different hidden layer sizes: X in $\{1, 2, 3, 4, 5, 8, 10, 16, 32\}$. For each architecture, 5 trials were run with different random seeds to ensure statistical robustness.

Architecture Search: 64-X-3 Network Performance



Accuracy Convergence for Different Architectures



Analysis:

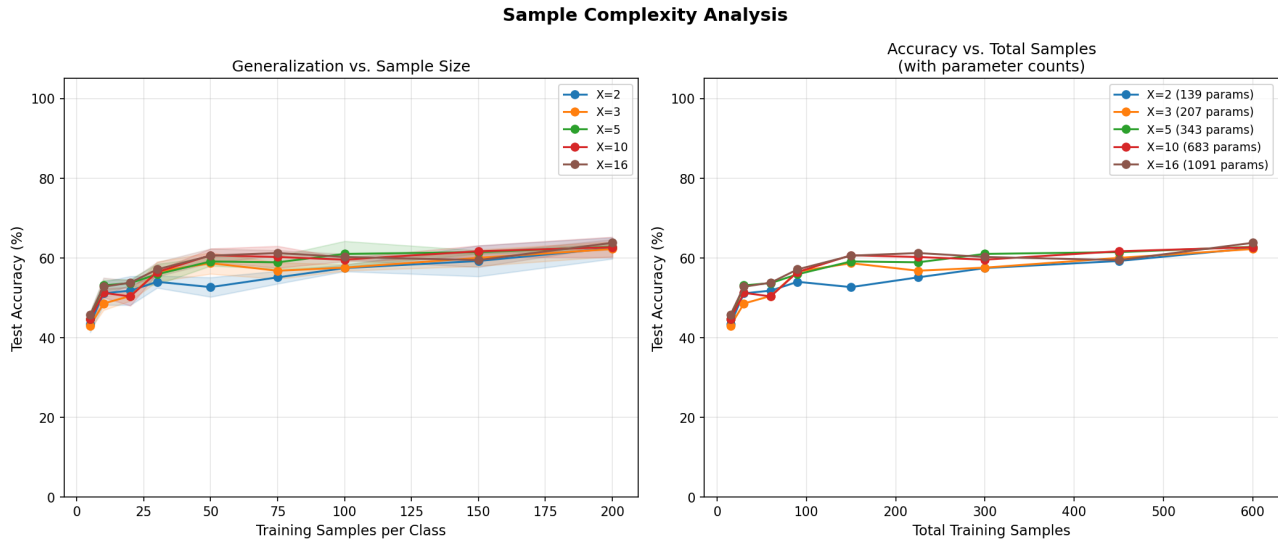
Key observations from the architecture search:

- $X=1$ and $X=2$ have too few hidden units to reliably separate 3 classes, resulting in poor or inconsistent accuracy.
- $X=3$ is the minimal viable architecture since 3 hidden units can create 3 distinct feature representations, one per class. Performance depends on initialization.
- $X=4$ to $X=8$ generally achieve reliable high accuracy with consistent convergence.

- $X=10$ and beyond provide additional capacity but may not significantly improve accuracy on this relatively simple task. However, more hidden units can improve convergence speed.
- There is a trade-off between model complexity (parameters) and performance. For this specific problem, the sweet spot appears to be around $X=5$ to $X=10$, offering high accuracy with reasonable model complexity.

8. Part (e): Sample Complexity Analysis

To study sample complexity, we varied the number of training samples per class (5 to 200) and measured generalization performance on a separate test set of 300 samples. This was done for multiple architectures to show how model capacity and data requirements interact.



Analysis:

The sample complexity analysis reveals several important insights:

- With very few samples (5-10 per class), all architectures struggle to generalize, regardless of the number of hidden units.
- Simpler architectures (X=2, X=3) require fewer samples to reach peak performance but have a lower performance ceiling.
- More complex architectures (X=10, X=16) require more training data to fully leverage their capacity, but can ultimately achieve higher accuracy.
- The results align with statistical learning theory: more complex models (higher VC dimension) require more training samples to achieve good generalization. The VC dimension grows with the number of parameters, so larger networks need proportionally more data.
- For this task with uniform noise in $[-5, +5]$ (SNR ~ -14 dB), approximately 30-50 samples per class are sufficient for reliable classification, demonstrating the neural network's ability to extract signal from very noisy data.

9. Conclusion

This assignment demonstrated the complete pipeline of training a neural network for pattern classification:

1. Data generation with controlled noise to simulate real-world variability.
2. Neural network training with backpropagation, showing that even a minimal 64-3-3 architecture can learn to classify noisy patterns.
3. Weight visualization provided interpretable insights into what features the network learned (input-hidden weights as spatial feature detectors, hidden-output weights as class decision rules).
4. A web application was built for practical deployment and testing.
5. Architecture search revealed the trade-off between model complexity and performance, with X=5-10 being the sweet spot for this task.
6. Sample complexity analysis confirmed theoretical expectations about the relationship between model capacity, training data size, and generalization.